CodeLens

Product Requirements Document

Understand a pull request before reviewing it

1. The Problem

Code reviews become harder as pull requests grow in size and complexity. Before a reviewer can meaningfully inspect the code, they often need to figure out what changed, how broad the change is, what areas may be affected, and where the highest-risk parts are.

This information is often spread across the diff, repository context, existing development tools, and the reviewer's own investigation.

The opportunity is to give engineers a fast, structured view of a pull request before they begin a detailed manual review.

2. The Core Idea

CodeLens is a GitHub-based tool that automatically analyzes a pull request and surfaces the signals a reviewer needs to understand the change.

The goal is not to replace code review. The goal is to make human review faster and more focused by answering:

What changed, how risky is it, and where should I look more closely?

3. Who Is It For?

- Developers reviewing pull requests
- Developers seeking early feedback on their own changes
- Tech leads and engineering managers who want visibility into change risk

4. Core Scope

The solution should explore how a PR can be analyzed across meaningful dimensions. These areas define the intended scope, but teams should decide how they are measured and presented.

- Change and drift: How much has changed, and does the change appear broader than its stated purpose?
- Code volume: How large is the PR relative to the repository or normal changes?
- Functionality: Does the PR appear to address one coherent piece of functionality or several?
- Critical functionality: Does the change affect areas such as authentication, payments, permissions, data handling, infrastructure, or other business-critical paths?
- Code quality: Are there meaningful maintainability, complexity, readability, or error-handling concerns?

- Security: Are there potential vulnerabilities, unsafe patterns, secrets, dependency risks, or security-sensitive changes?
- Test coverage: Are the important parts of the change adequately covered by tests?

5. GitHub Context

- A reviewer should be able to use CodeLens in the context of a GitHub pull request.
- The analysis should consider the PR diff and relevant repository context.
- Results should be understandable within the normal review workflow.
- The experience should avoid creating unnecessary noise for authors or reviewers.

6. Expected Outcome

Build a working prototype that demonstrates how CodeLens could help an engineer understand a PR before performing a detailed manual review.

The prototype should make it possible to see the important signals from a change and understand why those signals matter.

The exact interface, scoring system, analysis approach, AI usage, integrations, and technical architecture are intentionally open.

7. Constraints and Non-Goals

- GitHub is the environment for the initial concept.
- Human review remains the final decision-maker.
- Do not build an automatic approval or merge system.
- Do not optimize for generating the largest number of findings.
- Prioritize useful, explainable signals over generic AI-generated prose.
- Do not try to replace every existing developer or security tool.
- Keep the first experience focused enough to be useful during an actual PR review.

8. What Teams Should Decide

Teams are expected to make product and technical decisions around questions such as:

- What should a reviewer see first?
- Which signals deserve the most attention?
- How should risk or confidence be represented?
- What evidence should accompany a finding?
- When should CodeLens stay quiet?
- How much repository context is necessary?
- How should AI and existing developer tools work together?
- Where should the experience live within GitHub?

9. Success Criteria

- A clear understanding of the code-review problem.
- A focused experience that helps reviewers orient themselves quickly.
- Useful and explainable analysis rather than unsupported claims.
- Thoughtful prioritization of risk and reviewer attention.
- A working prototype that makes the value of CodeLens easy to understand.

10. The Challenge

Design and build a product that helps an engineer go from:

"I have a large pull request. Where do I start?"

to:

"I understand this change, I know what looks risky, and I know where to focus my review."

Build the version of CodeLens that you believe would make code review meaningfully better.

11. Reference

For reference, participants can refer to CodeRabbit and take inspiration.